

Entorno personal de aprendizaje:

Programación Básica en Python



Ministerio de
**TECNOLOGÍAS
DE LA INFORMACIÓN
Y COMUNICACIÓN**

 **GOBIERNO
NACIONAL**

*Paraguay
de la gente*

Unidad 2.**Elementos de programación en Python**

MINISTERIO DE TECNOLOGÍAS DE LA INFORMACIÓN Y COMUNICACIÓN

Viceministerio de Tecnologías de la Información y Comunicación

Dirección General de Inclusión Digital y TIC en la Educación

Información

SEDE 1



Ministerio de
**TECNOLOGÍAS
DE LA INFORMACIÓN
Y COMUNICACIÓN**

Dirección: Complejo Santos E2 - Gral. Santos 1170 c/ Concordia

Teléfono: (+595 21) 217 9000

Correo electrónico: comunicacion@mitic.gov.py

Mesa de entrada: sgeneral@mitic.gov.py

Asunción – Paraguay

Elementos de programación en Python

Elementos de programación en Python

1. Tipos de datos y conversiones de datos en Python

Una máquina solamente puede entender secuencias de código binario (“0” y “1” en diversas combinaciones), pero eso no significa que debamos escribir siempre este tipo de secuencias; justamente, el lenguaje de programación (y a partir de aquí nos centraremos específicamente en Python) provee de representaciones para estos conjuntos de valores, según se vaya necesitando. A estas representaciones llamamos “tipo de datos”.

El objetivo de una computadora es el manejo de datos o información. Un *dato* es la expresión general que describe los objetos con los cuales trabaja una computadora; los algoritmos y programas operan sobre estos datos, que pueden ser de varios tipos. Según el tipo de datos, la máquina reserva mayor o menor espacio de trabajo en su capacidad de memoria, de modo a optimizar el trabajo. En concreto, **un tipo de datos** es un conjunto específico de valores de datos y a su vez un conjunto de operaciones que actúan sobre estos datos¹.

Los tipos de datos básicos o primitivos en Python son:

- ✓ Numéricos: **int**, enteros; **float**, de coma flotante
- ✓ Texto (cadenas): **str**
- ✓ Lógicos (booleanos): **bool**

“**int**” representa números enteros, aquellos que no tienen componentes fraccionarios ni decimales, y a su vez pueden ser positivos o negativos. Sirve como ejemplo para representar números como edades, población, cantidad de familiares.

“**float**” es una representación de números con decimales, es decir, de coma flotante. Por ejemplo, el promedio de un conjunto de valores, la temperatura dada de un cuerpo, una calificación de un examen-

“**str**” representa una cadena de caracteres en sucesión. Para ejemplificar, una cadena que contenga el nombre y apellido completo de una persona, o la descripción de un lugar.

“**bool**” representa el tipo “lógico” de datos: verdadero (*True*) o falso (*False*). En este caso este claro: se cumple alguna condición o no, por ejemplo, para establecer si aprobamos algún curso, o si una persona es mayor de edad ante la ley.

Unidad 2.

Elementos de programación en Python

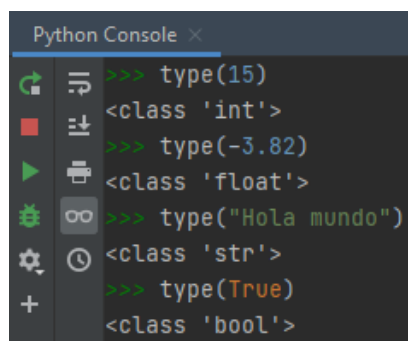
En un programa en Python o en una consola (que sirve para ejecutar comandos básicos) se puede tipear el comando `type` con la siguiente sintaxis:

`type (dato)`

Y se devolverá el tipo de dato en el siguiente formato:

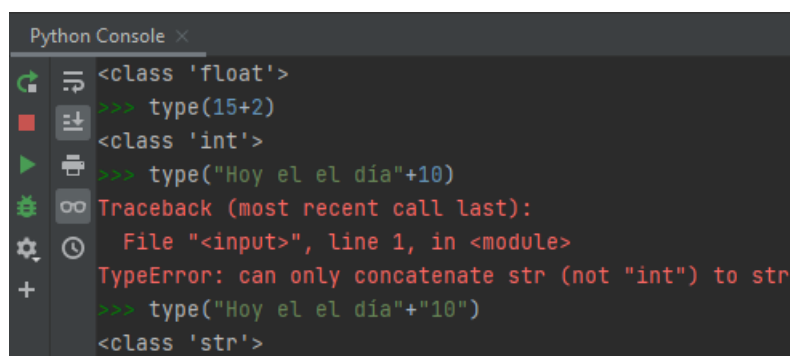
`<class 'tipo del dato'>`

Por ejemplo, si ejecutamos este comando sobre distintos tipos de datos: entero, flotante, cadena (las cadenas de texto se ingresan entre comillas dobles o bien simples, pero siempre encerradas entre ellas), o booleano obtendremos:



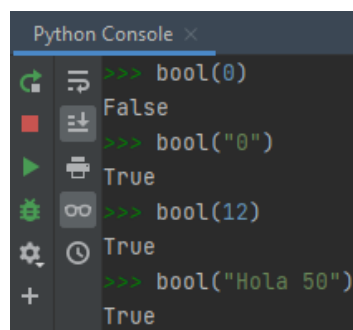
```
Python Console x
>>> type(15)
<class 'int'>
>>> type(-3.82)
<class 'float'>
>>> type("Hola mundo")
<class 'str'>
>>> type(True)
<class 'bool'>
```

Cuando trabajamos con distintos tipos de datos, debemos tener algunos cuidados para operarlos. Sin adentrarnos en definiciones sobre expresiones y operadores, cuando hacemos alguna operación que involucre a tipos distintos de datos, debemos tener en cuenta la compatibilidad; más adelante nos adentraremos en ejemplos más complejos, pero para referencia, podemos trabajar con los tipos numéricos entre sí, pero al querer mezclarlos con cadenas de texto, tendremos un error, que formalmente resolveremos al detallar operadores y expresiones. Por otro lado, las variables de tipo *bool* son las más sencillas de plantear: se evalúa el argumento entre paréntesis y cualquier valor que no sea el número 0 o bien el espacio vacío entre comillas "" dará verdadero (*True*); los dos citados antes devolverán falso (*False*). Van aquí varios ejemplos de tipos de datos devueltos por la consola en Python:



```
Python Console x
<class 'float'>
>>> type(15+2)
<class 'int'>
>>> type("Hoy el el día"+10)
Traceback (most recent call last):
  File "<input>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
>>> type("Hoy el el día"+"10")
<class 'str'>
```

Unidad 2. Elementos de programación en Python

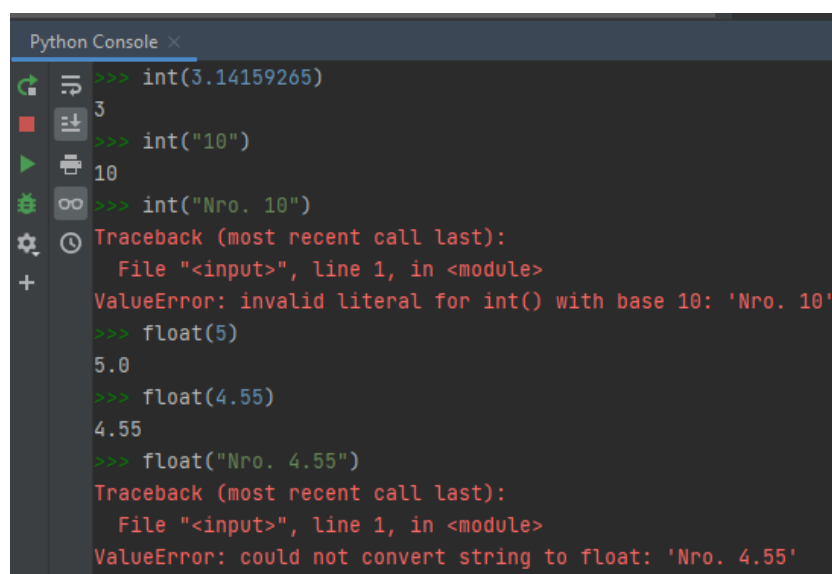


```
Python Console x
>>> bool(0)
False
>>> bool("0")
True
>>> bool(12)
True
>>> bool("Hola 50")
True
```

Si queremos convertir un número a cadena, lo podemos hacer utilizándolo como argumento de una cadena formal; con la palabra `str()`, y poniendo dentro del paréntesis cualquier cadena que queramos, incluido cómo no un número, como vemos en el ejemplo de abajo.

```
>>> "Hoy es el día "+str(10)
'Hoy es el día 10'
```

Finalmente, también se pueden cambiar tipos de datos si se considera necesario, dado el caso, poniendo el tipo de dato deseado y trabajando con el argumento según sea número o texto, y siempre que un “texto” sea válido como número, como se ve en los ejemplos:



```
Python Console x
>>> int(3.14159265)
3
>>> int("10")
10
>>> int("Nro. 10")
Traceback (most recent call last):
  File "<input>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'Nro. 10'
>>> float(5)
5.0
>>> float(4.55)
4.55
>>> float("Nro. 4.55")
Traceback (most recent call last):
  File "<input>", line 1, in <module>
ValueError: could not convert string to float: 'Nro. 4.55'
```

2. Operadores y Expresiones

Una vez que sabemos cómo representar valores mediante tipos de datos apropiados, tenemos que decir a la computadora qué debe hacer con esos valores. Por ejemplo, para razonar con algo sencillo, si dados dos datos queremos que la máquina devuelva la suma de ellos (como se vio en algunos ejemplos previos) pues entonces utilizamos el operador “suma” entre los dos valores.

Un **operador** es cualquier símbolo ya sea aritmético, de comparación, lógico, o bien de

Unidad 2.

Elementos de programación en Python

operación de cadenas, que opera sobre variables. Los operadores más importantes son:

- Aritméticos:** suma (+), resta o inverso aditivo (-), multiplicación (*), división (/), división entera (//), módulo resto (%), exponenciación (**)
- Comparativos:** menor (<), menor o igual (<=), igual (==), mayor (>), mayor o igual (>=), distinto (!=)
- Lógicos:** y (and), o (or), no (not)
- Cadenas:** + (concatena cadenas de texto), * (multiplica/repite una cadena de texto)

Tenemos datos que pueden ser constantes en un programa dado, otros variables o sea cuyo valor puede cambiar a lo largo del mismo. A la combinación de constantes, variables, símbolos de operación, paréntesis y funciones especiales, se denomina **expresiones**.

Son ejemplos de expresiones operadores aritméticos sobre tipos de datos **int** y **float**:

```
>>> 5+3
8
>>> 5-3
2
>>> 5*3
15
>>> 5/3
1.6666666666666667
```

```
>>> -5 #inverso aditivo
-5
>>> 5**3 #exponenciacion
125
>>> 5//3 #division entera
1
>>> 5%3 #resto división
2
```

(al insertar un caracter numeral "#", lo que viene después es un *comentario* de programa en Python, es decir, el programa directamente lo ignora, y puede ser útil para etiquetar secciones de código, o poner comentarios relevantes)

Cuando tenemos más de un operador en una misma expresión, esta se evalúa siempre de acuerdo con la precedencia de los operadores, es decir la prioridad que tienen. Y cuando tenemos operadores con la misma precedencia, estos se evalúan de acuerdo con su regla de asociatividad. Estas reglas son básicamente las mismas que manejamos desde nuestros cursos básicos de matemática. Si utilizamos los operadores de agrupación, típicamente conocidos como paréntesis, podemos dar prioridad a ciertas expresiones que se evalúan siempre en primer lugar y siempre de adentro hacia afuera.

Por ejemplo para evaluar la siguiente expresión: $(3+5//4-2)-2**4+3*(7-2)$

```
>>> (3+5//4-2)-2**4+3*(7-2)
1
```

Unidad 2.

Elementos de programación en Python

El resultado, como se ve, es 1. Ahora bien, la precedencia indicaba que el resto de dividir $5//4$ (división entera) se debía sumar algebraicamente con los demás componentes del primer paréntesis; el término $2**4$ (2 elevado a la cuarta potencia) debe evaluarse primero para contribuir a la suma algebraica de las demás operaciones, y que 3 se debe multiplicar por la operación del paréntesis $(7-2)$ para poder contribuir también a la suma general.

A su vez, los operadores de comparación realizan valga la redundancia la comparación adecuada, y devuelven siempre un valor de tipo *bool*; de ser verdadera, devuelven como resultado el valor booleano *True*; en caso contrario devuelven *False*.

```
>>> 2<2.55
True
>>> 3>=10
False
>>> 4!=8
True
>>> 6==9
False
```

Los operadores lógicos (*and*, *or*, *not*) se aplican sobre valores de tipo *bool* (que bien pueden ser el resultado de las expresiones comparativas como se vio antes), y también devuelven siempre un valor de tipo *bool*.

```
>>> 3>10 and 2<8 #conjunción (Y)
False
>>> 3>10 or 2<8 #disyunción (O)
True
>>> not 3>10 #negación (no)
True
```

La tabla de asociatividad referencial es:

Operador	Asociatividad	Ejemplo	Resultado
**	Derecha a izquierda	$2**4**2$	65536
+, - (unarios)	Ninguna	$-3**3$	-27
*, /, //, %	Izquierda a derecha	$*10/2*3$	15
+, - (binarios)	Izquierda a derecha	$2-5+6$	3

Para cadenas de texto, lo que puede aplicarse (aparte de operaciones comparativas, pero no abordaremos este tema en el curso por ser más profundo que el objetivo de aprendizaje) son los operadores de concatenación (sirven para unir cadenas) y repetición (las repite según la cantidad indicada) o bien una combinación entre ellos:

Unidad 2.

Elementos de programación en Python

```
>>> "Hoy es "+"Lunes"
'Hoy es Lunes'
>>> "Lu"*4
'LuLuLuLu'
>>> "Hoy es "+"Lu"*4
'Hoy es LuLuLuLu'
```

3. Variables y asignación de valores

Hemos mencionado antes a la ligera que los programas de computadora contienen valores que no van a cambiar durante la ejecución; tales valores son las *constantes*.

Por otro lado, existen otros valores que se espera que cambien durante la ejecución del programa, para poder cumplir los propósitos del mismo; a estos se les llaman **variables**. Las mismas pueden ser de uno de los tipos de datos contenidos en el lenguaje Python: enteras, flotantes, cadenas, booleanas.

Una variable de un cierto tipo de datos, solamente puede contener o tomar valores de ese tipo; como vimos antes, una variable de cadena de caracteres, sólo puede tomar caracteres y no números; lo mismo para las variables numéricas y en general para cualquier variable distinta, salvo los casos de conversión de tipo de datos antes estudiados. Si se intenta asignar a una variable de un tipo un valor de otro tipo, se producirá un error de tipo de datos.

En general, las variables nos sirven para almacenar también resultados de expresiones que debemos preservar para algún cálculo futuro o para obtener ciertos valores trabajados que nos permitan cumplir el propósito de un programa en cuestión; **el símbolo de igualdad simple (=) nos permite asignar** a la variable presente a su izquierda, cualesquiera valores o expresiones queden a su derecha (no confundir con la “**igualdad doble**” ==, que implica lógica comparativa). La variable debe tener un nombre, el cual tiene que seguir ciertas reglas que veremos también. Pero por de pronto, la sintaxis de asignación es

$$\text{nombre_variable} = (\text{expresión})$$

Por ejemplo, si queremos almacenar en una variable, la masa de una persona, y en otra su altura, para calcular luego índice de masa corporal (IMC) podemos hacerlo como sigue:

```
>>> masa=65 #en Kg
>>> altura=1.70 #en metros
>>> masa/(altura**2)
22.49134948096886
```

Unidad 2.

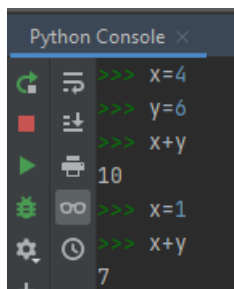
Elementos de programación en Python

La cuestión es que si escribo un programa que automatice esto, puedo hacer que dado el ingreso desde teclado de dos valores, como son la masa y la altura de una persona cualquiera, pueda calcular el IMC, no estando limitado específicamente a “una” altura y a “una” masa.

Ahora bien, hay que tener algunas consideraciones especiales con las variables: sus nombres o *identificadores* **deben cumplir ciertas reglas**:

- ✓ Deben empezar con una letra o bien un guión bajo `_`
 - Ejemplos: A1, Base, `_d`
- ✓ Pueden continuar el nombre con letras, números o `_`
 - Ejemplos: B_1, c1, `_1A`
- ✓ Los identificadores distinguen entre mayúsculas y minúsculas
 - Ejemplos: Base, BASE, BAse, basE
- ✓ No pueden ser NUNCA una palabra reservada del sistema, por ejemplo, `and`, `or`, `not`, `if`, `else`, `for`, `True`, `False` y otras que iremos viendo en el desarrollo del curso
 - Ejemplos INCORRECTOS: `if=6`, `and=0`, `or=5`, `True=10`

Una vez que la variable se utilizó, perfectamente puede ser reasignada para otros usos.



(se ve que en primera instancia, `x=4` e `y=6`, por lo cual su suma despliega el resultado “10”; luego se modifica el valor de `x` únicamente al nuevo valor “1”, y la misma expresión de suma `x+y` da otro resultado esta vez: “7”, para los nuevos valores)

4. Impresión de valores y variables en pantalla

A partir de este punto, nuestro análisis un tanto “teórico” para presentar las bases de la programación y del lenguaje, darán paso a un enfoque más práctico; a programar únicamente se aprende programando.

Cuando un algoritmo resuelve lo que necesitamos, usualmente queremos desplegar ese cálculo en la pantalla, o bien imprimir algún mensaje, etc.

Unidad 2.

Elementos de programación en Python

Esto lo podemos hacer con el comando *print*, y la sintaxis que se muestra:

print (*expresión*)

Hasta aquí, habíamos usado la consola de Python para cálculos básicos o impresiones de cadenas; pero realmente no estudiamos a fondo la “estructura” de un programa en Python. Y, justamente, hablábamos de que una de sus principales virtudes es la simplicidad y facilidad. Pues bien, como vimos en el final del módulo anterior, esta siguiente línea que verán a continuación es nuestro primer programa formal en Python:

```

>>> print ("Mi primer programa en Python")
Mi primer programa en Python

```

Es un programa de una única línea, que solamente imprime textualmente la frase dada, sin hacer ningún cálculo ni operación adicional sobre ella. Como se ve, muestra al máximo la simplicidad de Python, cuando en otros lenguajes la misma acción requeriría de más líneas de código. Claro está que tanto en Python como en cualquier otro lenguaje de programación no hay una única forma de llegar a la resolución de un problema, se pueden emplear varios caminos; por ejemplo, el mismo programa (con el *mismo* resultado), puede escribirse de la siguiente manera usando cadenas de texto:

```

mensaje="Mi primer programa en Python"
print(mensaje)

```

```

>>> mensaje="Mi primer programa en Python"
>>> print(mensaje)
Mi primer programa en Python

```

Se pueden imprimir expresiones de cualquier tipo, como las que estuvimos utilizando en ejemplos anteriores:

```

>>> print((3+5//4-2)-2**4+3*(7-2))
1
>>> fahrenheit=91
>>> celsius=(fahrenheit-32)*5/9
>>> print(celsius)
32.77777777777778
>>> temperatura=25
>>> print (temperatura>20)
True

```

También se pueden hacer líneas un poco más complejas, como por ejemplo impresión de varias variables, en una misma línea de comando.

```

>>> mensaje1="Soy"
>>> mensaje2="Juan Pérez y tengo"
>>> edad=35
>>> mensaje3="años"
>>> print(mensaje1,mensaje2,edad,mensaje3)
Soy Juan Pérez y tengo 35 años

```

Unidad 2. Elementos de programación en Python

5. Ingreso de datos a programas

De forma similar al comando estudiado recién, cuando uno quiere dirigir el flujo de datos en sentido inverso, es decir, *ingresar* valores desde un periférico de entrada (en nuestro caso, un teclado) hacia la computadora o programa, hay un comando que permite hacerlo, el *input*, cuya sintaxis es la siguiente:

```
variable = input (texto opcional)
```

Es importante tener en cuenta que **siempre** se obtiene con este comando un valor de tipo cadena (**str**), por lo cual, si quisiéramos ingresar un valor numérico o incluso booleano, debemos hacer uso de las conversiones de tipos de datos que hemos analizado antes, las cuales por otra parte no son complicadas. Además, el argumento es opcional (o sea, los paréntesis pueden quedar vacíos) y solamente sirven para desplegar un texto orientativo en pantalla antes de ingresar el valor en cuestión.

```
>>> nombre=input("¿Cómo te llamas?")
¿Cómo te llamas?>> David
>>> edad=int(input())
>> 30
>>> print(nombre,"tiene",edad,"años.")
David tiene 30 años.
```

Mucho cuidado con las conversiones numéricas (int o float), si vamos a usar la variable para otro propósito que no sea imprimir mensajes, deberemos convertirlas como en el código de arriba, pues si no puede arrojar errores como el siguiente caso:

```
>>> edad=input()
>> 30
>>> print(nombre,"tiene",edad+1,"años.")
Traceback (most recent call last):
  File "<input>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
```

Es un error de tipo de datos, y para solucionarlo, debimos convertir el ingreso (input) aplicando un cambio de tipo *int* o *float* según convenga dado el caso, usando ese comando de ingreso como su argumento.

```
>>> edad=int(input())
>> 30
>>> print(nombre,"tiene",edad+1,"años.")
David tiene 31 años.
```

Unidad 2. Elementos de programación en Python

6. Uso del laboratorio virtual de programación VPL

El *Campus Virtual* del MITIC tiene integrado un laboratorio virtual de programación (*Virtual Programming Lab*, o VPL) que hace las veces de entorno de desarrollo integrado, listo para que podamos practicar ejercicios y evaluarlos de forma automática con desafíos de programas que los docentes consideran relevante para tu aprendizaje.

Si bien el VPL ofrece la posibilidad de usarlo como un editor de código simple, su uso principal radica la corrección automática que brinda a los programas. Veremos con un ejemplo las posibilidades, con una “Actividad 1” de prueba; de forma que sirva de introducción a este tipo de evaluaciones. También, como apoyo, hay un video sobre este mismo proceso en la sección correspondiente de material.

Primero, hay que ingresar a la actividad, en este ejemplo la 1.1 (las actividades del VPL tienen un ícono de computadora al lado). A continuación hay que hacer clic en el botón “Editar”:



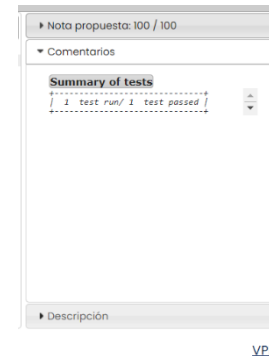
Ahora ya estaremos en el editor de código, que puede estar vacío o completado con algunas líneas, dependiendo del programa en cuestión. Aquí deberemos completar el programa de la forma en que creemos es la correcta. Una vez completado el programa, le damos clic al botón de guardar (tiene forma de un diskette antiguo). Finalmente, evaluamos el código haciendo clic en el botón central que tiene un ícono de “check” en el centro, seguido de un número (suele empezar en cero).



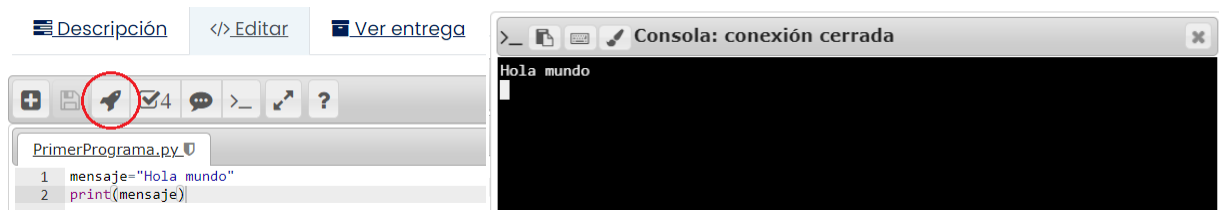
Unidad 2.

Elementos de programación en Python

En la zona derecha veremos una sección con la calificación automática de acuerdo con las consignas del problema y los casos exitosos según la programación del docente, y en caso de error, veremos descripciones del o los errores.



Finalmente, en la herramienta VPL también podemos ejecutar el código escrito. Para ello, se debe hacer clic en el ícono de “cohete” (*Ejecutar*) que está entre los botones de *Guardar* y *Evaluar*. Esto ejecuta el código tal cual se ejecutaría en una consola de Python, como por ejemplo en PyCharm Community, y se despliega una ventana mostrando el resultado de la ejecución.



Con esto termina el análisis de la herramienta VPL, el cual es complementado con el vídeo de apoyo correspondiente, como se indicó. Si bien no es una herramienta tan potente como el IDE de PyCharm u otro similar en cuanto a procesamiento de código, sí se puede usar también el VPL como un ambiente de desarrollo de Python.

Referencias

Joyanes Aguilar, L. (2008). Fundamentos de Programación. McGraw-Hill

Síntesis de material propio por parte de los Docentes.